

Advanced Data Management

Query Answering in (G)LAV Materialization

Domenico Fabio Savo

*Corso di laurea magistrale
INGEGNERIA INFORMATICA
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione
University of Bergamo*

Outline of the course

1. Introduction to propositional logic
 2. Introduction to First-order Logic
 3. Relational Calculus
 4. Information Integration **S**ystems (IIS)
 5. Logical formalization of IISs
 6. Mapping between **Global Schema** e **Data Sources**
 7. Incomplete information databases
 8. **Query answering over IISs**
 9. **O**ntology-**B**ased **D**ata **M**anagement (OBDM)
 10. Description Logic Ontologies
 11. Query answering in ontologies
 12. Query answering in OBDM
-

Query Answering in (G)LAV

Logical Formalization of Information Integration Systems – Syntax

An information integration system (IIS) is a triple $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ where:

- $\mathbf{G}$ is the **global schema**
 - A logical theory over a relational alphabet A_G .
- $\mathbf{S}$ is the **source schema**
 - A logical theory over a relational alphabet A_S .
- $\mathbf{M}$ is the **mapping between $\mathbf{S}$ and $\mathbf{G}$**
 - A logical theory over the relational alphabet $A_S \cup A_G$.

Logical Formalization of I.I.S.s – Semantics

Assume a information integration system $\langle \mathbf{J}, \mathbf{D} \rangle$, where $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ is an information integration specification and $\mathbf{D}$ is a **source database** for $\mathbf{S}$

The semantics of $\langle \mathbf{J}, \mathbf{D} \rangle$ is defined as follows:

$$\text{sem}(\mathbf{J}, \mathbf{D}) = \{ \mathbf{B} \mid \mathbf{B} \text{ is a global database s.t. } (\mathbf{B}, \mathbf{D}) \models \mathbf{M} \}$$

Assume a query q over A_G . The **certain answers** of q w.r.t. $\mathbf{J}$ and $\mathbf{D}$ are:

$$\text{cert}(q, \mathbf{J}, \mathbf{D}) = \{ \bar{a} \mid \bar{a} \in q(\mathbf{B}) \text{ for each } \mathbf{B} \in \text{sem}(\mathbf{J}, \mathbf{D}) \}$$

Query Answering – Recognition Problem

- **DEF:** **QA-IIS** is the following decision problem:
 - **Input:** an I.I.S. $J = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$, a FOL query $q(\bar{x})$, a source database D , a tuple of constants $\bar{c}$
 - **Question:** Is $\bar{c} \in \text{cert}(q, J, D)$?

Conjunctive Query (recap)

- Let $\varphi(x_1, \dots, x_n)$ be a FOL formula with free variables $x_1, \dots, x_n$
- $\varphi(x_1, \dots, x_n)$ is an Existential Conjunction if it is of the form

$$\exists y_1, \dots, \exists y_n P_1(\bar{z}_1) \wedge \dots \wedge P_k(\bar{z}_k)$$

- A FOL query $\{(x_1, \dots, x_n) | \varphi(x_1, \dots, x_n)\}$ is a **Conjunctive Query** if $\varphi(x_1, \dots, x_n)$ is an Existential Conjunction.

Union of Conjunctive Queries (recap)

- A formula $\varphi(x_1, \dots, x_n)$ is Union of Existential Conjunctions if it is of the form

$$\bigvee_i \psi_i(x_1, \dots, x_n)$$

where each ψ_i is an existential conjunction.

Observe: same free variables.

- A FOL query $\{(x_1, \dots, x_n) \mid \varphi(x_1, \dots, x_n)\}$ is a **Union of Conjunctive Queries** if $\varphi(x_1, \dots, x_n)$ is a Union of Existential Conjunctions.

Our Assumptions

As made for GAV, we assume:

- That **G** is an empty theory over the alphabet A_G , i.e., **G** is a database schema without integrity constraints.
- That **S** is an empty theory over the alphabet A_S , i.e., **S** is a database schema without integrity constraints.
- That **GLAV mappings** are defined via queries that we can actually handle: **Unions of Conjunctive Queries**

From now on, if not otherwise stated, we always consider the assumptions made above to be true moreover, if not otherwise stated, we will assume that the **user queries are **conjunctive queries****

Local As View (recap)

LAV mappings describe the **source schema** in terms of the **global schema**

A LAV **mapping assertion** is a pair $\langle s, q_g \rangle$ where:

- s is a **single predicate** name of arity n in A_S
- $q_g = \{\bar{x} \mid \varphi_g(\bar{x})\}$ is a FOL **query of arity n over A_G**

Logical formalization of LAV mapping assertions:

$$\forall \bar{x} . s(\bar{x}) \rightarrow \varphi_g(\bar{x})$$

A LAV mapping is a **finite** set of LAV mapping assertions

Global and Local As View (recap)

Materialization techniques for **LAV** mapping are similar to those for **GLAV**, so we will look at the more general case of GLAV

A GLAV **mapping assertion** is a pair $\langle q_s, q_g \rangle$ where:

- $q_s = \{\bar{x} \mid \varphi_s(\bar{x})\}$ is a FOL query of arity n over A_s
- $q_g = \{\bar{x} \mid \varphi_g(\bar{x})\}$ is a FOL query of arity n over A_g

Logical formalization of GLAV mapping assertions:

$$\forall \bar{x}. \varphi_s(\bar{x}) \rightarrow \varphi_g(\bar{x})$$

A GLAV mapping is a **finite** set of GLAV mapping assertions.

Conjunctive GLAV Mapping

A **Conjunctive GLAV mapping assertion** is a pair $\langle q_s, q_g \rangle$ where:

- $q_s = \{\bar{x} \mid \varphi_s(\bar{x})\}$ is a **CQ** of arity n over A_s
- $q_g = \{\bar{x} \mid \varphi_g(\bar{x})\}$ is a **CQ** of arity n over A_g

A **Conjunctive GLAV mapping** is a **finite** set of Conjunctive GLAV mapping assertions.

Conjunctive Mapping – Examples (1)

- Conjunctive **LAV** mapping

$$\forall x, y. \textit{Tab1}(x, y) \rightarrow \exists z. \textit{Person}(x) \wedge \textit{Parent}(y) \wedge \textit{hasFather}(x, z)$$

- Conjunctive **GAV** mapping

$$\forall x. \forall y. \exists z. \textit{Tab1}(x, y) \wedge \textit{Tab2}(y, z) \rightarrow \textit{hasFather}(x, y)$$

- Conjunctive **GLAV** mapping

$$\forall x. \forall y. \exists z. \textit{Tab1}(x, y) \wedge \textit{Tab2}(y, z) \rightarrow \exists v. \textit{hasFather}(x, y) \wedge \textit{hasFather}(y, v)$$

Conjunctive Mapping – Examples (2)

- **Non-Conjunctive LAV** mapping

$$\forall x, y. \textit{Tab1}(x, y) \rightarrow \exists z. \textit{Person}(x) \wedge \textit{Parent}(y) \wedge \textit{hasFather}(x, z) \wedge \neg \textit{Woman}(z)$$


- **Non-Conjunctive GAV** mapping

$$\forall x. \forall y. \exists z. \textit{Tab1}(x, y) \wedge \textit{Tab2}(y, z) \wedge \neg \textit{Tab3}(z) \rightarrow \textit{hasFather}(x, y)$$


- **Non-Conjunctive GLAV** mapping

$$\forall x. \forall y. \textit{Tab1}(x, y) \rightarrow \textit{hasFather}(x, y) \vee \textit{hasMother}(x, y)$$


Query Answering – Recognition Problem

- Let **CQ-GLAV** be the class of **conjunctive GLAV mappings**
- **DEF: UCQ-QA-IIS($\mathcal{L}$)** where $\mathcal{L}$ is a class of mappings is the following problem
 - **Input:** An I.I.S. $J = \langle G, M, S \rangle$ where G and S are empty theories and M is a mapping in $\mathcal{L}$, a source database D , a UCQ q , and a tuple of constants $\bar{c}$
 - **Question:** Is $\bar{c} \in \text{cert}(q, J, D)$?
- **Thm: UCQ-QA(CQ-GLAV)** is **decidable**
 - Via materialization
 - Via virtualization

Query Answering In Information Integration

	Materialized	Virtual
GAV	1	2
LAV	3	4

Materialization in (G)LAV

Assume an I.I.S $J = \langle G, M, S \rangle$ and a **source database D** for **S**

As for GAV, we want to produce a database $M(D)$ to represents the result of the integration, i.e., a **global instance** representing all the **global databases** in $\text{sem}(J, D)$

Desired properties of the materializations.

- We want to use the materialization $M(D)$ to compute **certain answers** to queries
- We want the materialization to be as **compact** as possible

With (G)LAV we cannot simply materialize views from the **sources** since there is no direct correspondence between definitions in the mapping and **global predicates**. Moreover, **we need to deal with existential quantifiers on the right-hand side of mapping assertions!**

Materialization in (G)LAV

With (G)LAV we cannot simply materialize views from the sources

- There is no direct correspondence between definitions and global predicates.
- Missing information (existential quantification) in the global schema definition

Intuitively, to materialize GLAV mappings we need null values

A **global instance K** over **G** is like a **global database** over **G** except that it may contain also **labeled nulls** (we can think of such **labeled nulls** simply as placeholders for *unknown* constants).

We denote by $\text{Const}(K)$ (resp. $\text{Nulls}(K)$) the set of constants (resp. labeled nulls) occurring in K

With a slight abuse of notation, we let $\text{dom}(K) = \text{Const}(K) \cup \text{Nulls}(K)$

Materialization in LAV – Example

Source database: $D = \{R(a,b), R(b,c)\}$

Mapping: $M = \{\forall x, y. R(x,y) \rightarrow \exists z. S(x,z) \wedge S(z,y)\}$

Which of the following **global instances** would be a good materialization?

➤ Symbols x , y , and z denote **labeled nulls**

- $K_1 = \{S(a,x), S(x,b), S(b,d), S(d,c)\}$
- $K_2 = \{S(a,x), S(x,b)\}$
- $K_3 = \{S(a,x), S(x,b), S(a,y), S(y,b), S(b,z), S(z,c)\}$
- $K_4 = \{S(a,x), S(x,b), S(b,y), S(y,c)\}$
- $K_5 = \{S(a,x), S(x,b), S(b,x), S(x,c)\}$

We still do not know!

Homomorphisms

A new tool: the homomorphism

To prove the next results we will rely on the notion of **homeomorphism**

Homomorphisms are a class of functions between relational structures

- Relational Structures are, essentially, sets of relational atoms
- A special instance of what mathematicians call structure-preserving mappings, i.e., mappings that do not alter the shape of a structure “too much”

We borrow the notion of homomorphisms and tailor it to our needs.

- **Don't tell mathematicians!**

The notion of homomorphism

Let **A** and **B** be two database instances over the same schema

A **homomorphism** from **A** to **B** is a function h from $\text{dom}(\mathbf{A})$ to $\text{dom}(\mathbf{B})$ ($\text{dom}(\mathbf{A})$ and $\text{dom}(\mathbf{B})$ can contain both *constants* and *variables\labeled nulls*) satisfying the following properties:

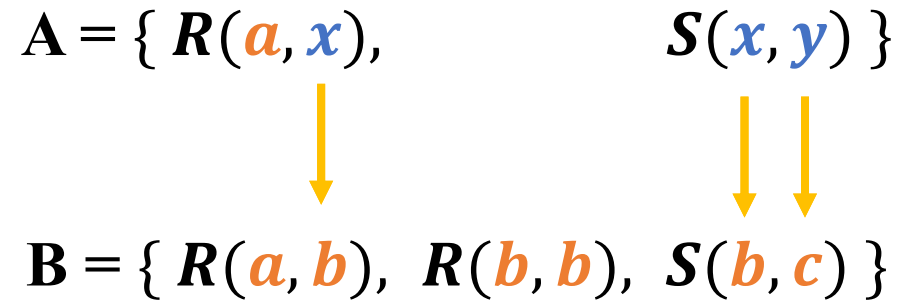
1. For every constant $c \in \text{Const}(\mathbf{A})$, $h(c) = c$
2. For every $R(\bar{a}) \in \mathbf{A}$, we have $R(h(\bar{a})) \in \mathbf{B}$

For a tuple $\bar{a} = (a_1, \dots, a_n)$ of constants and labeled nulls, $h(\bar{a}) = (h(a_1), \dots, h(a_n))$. In other words, condition 2. always ensures $h(\mathbf{A}) \subseteq \mathbf{B}$ whenever h is a **homomorphism** from **A** to **B**, where $h(\mathbf{A}) = \{R(h(\bar{a})) \mid R(\bar{a}) \in \mathbf{A}\}$

Homomorphisms – Example

Example: $\text{Const}(A) = \{a\}$ and $\text{Nulls}(A) = \{x,y\}$

$\text{Const}(B) = \{a,b,c\}$ and $\text{Nulls}(B) = \{\}$



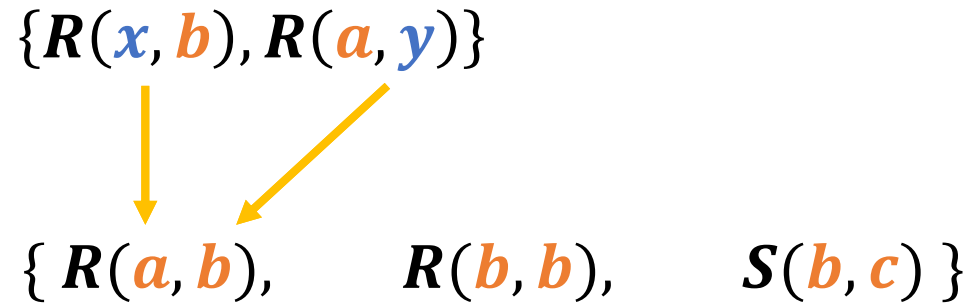
The following function h defines a homomorphism from A to B

- $h(a) = a$
- $h(x) = b$
- $h(y) = c$

Homomorphisms – Example

Example: $\text{Const}(A) = \{a, b\}$ and $\text{Nulls}(A) = \{x, y\}$

$\text{Const}(B) = \{a, b, c\}$ and $\text{Nulls}(B) = \{\}$



The following function h defines a homomorphism

- $h(a) = a, h(b) = b$
- $h(x) = a$
- $h(y) = b$

Homomorphisms – Example

Example: $\text{Const}(A) = \{a,b\}$ and $\text{Nulls}(A) = \{x,y\}$

$\text{Const}(B) = \{a,b,c\}$ and $\text{Nulls}(B) = \{\}$

$\{ R(a, x), \quad S(x, b), \quad S(x, y) \}$

$\{ R(a, b), \quad R(b, b), \quad S(b, c) \}$

There is no homomorphisms here!

Exercise on Homomorphisms

For which of the following there exists a homomorphism? In **red** the labeled nulls

1. $A = \{R(a,b)\}$

$B = \{R(a,b)\}$

2. $A = \{R(a,\mathbf{x})\}$

$B = \{R(c,d)\}$

3. $A = \{R(a,\mathbf{x}), S(\mathbf{x},c)\}$

$B = \{R(a,b), R(d,c)\}$

4. $A = \{R(a,\mathbf{x})\}$

$B = \{R(a,b), R(b,c)\}$

Exercise on Homomorphisms - Solution

For which of the following there exists a homomorphism? In **red** the labeled nulls

1. $A = \{R(a,b)\}$

– Yes. Identity.

$B = \{R(a,b)\}$

2. $A = \{R(a,\mathbf{x})\}$

– No.

$B = \{R(c,d)\}$

3. $A = \{R(a,\mathbf{x}), S(\mathbf{x},c)\}$

– No.

$B = \{R(a,b), R(d,c)\}$

4. $A = \{R(a,\mathbf{x})\}$

– Yes, $h(x) = b$.

$B = \{R(a,b), R(b,c)\}$

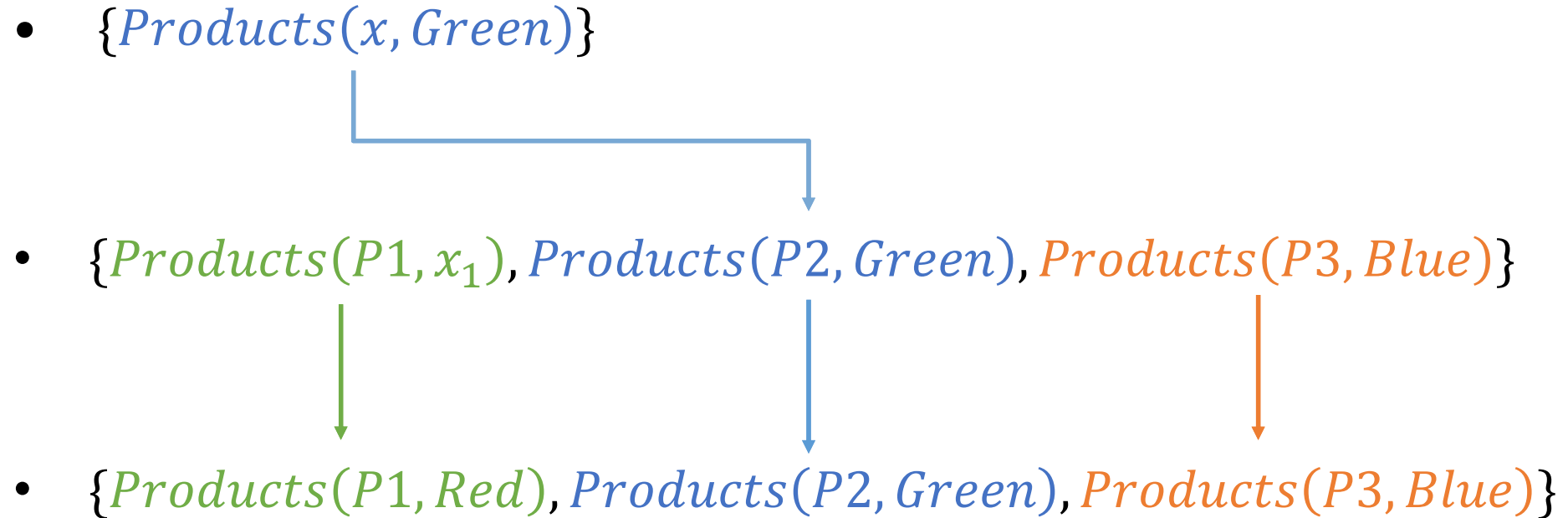
Composition of Homomorphisms

Let h be an homomorphism from A to B , and let h' be an homomorphism from B to C . The **composition** $h' \circ h$ of h, h' is the mapping defined as follows: $h' \circ h(x) = h'(h(x))$

Thm: The composition $h' \circ h$ is itself a homomorphism h'' from A to C .

Proof: Let $\alpha \in A$. Then, $h(\alpha) \in B$ and therefore $h'(h(\alpha)) \in C$. The claim follows straightforwardly.

Composition of Homomorphisms – Example



Finding Homomorphisms

Finding **homomorphisms** is harder than it may seem

- **Thm:** Given two sets of atoms A and B , checking whether there exists a homomorphism from A to B is NP-Complete
- **Thm:** Let A be a set of atoms. For an input set of atoms B , checking whether there exists a homomorphism from A to B is in LOGSPACE
 - Indeed, if we fix A , the number of possible assignments we need to test becomes small

Homomorphisms – Extensions

We can extend the notion of **homomorphisms** to different objects

- **Complete Databases.** Given an interpretation I , i.e., a complete database, we can naturally convert I into a set of atoms.
- **Incomplete Databases.** An incomplete database is a logical theory (set) of relational atoms.
- **Conjunctive Queries.** A conjunction of atoms can be seen as a set.

From now on, we will talk about **homomorphisms** in all the cases above

Query Answering and Homomorphisms (Complete Databases)

We have following well known property:

Let $q = \{\bar{x} \mid \bigvee_i \psi_i(x_1, \dots, x_n)\}$ be a UCQ and D be a database instance

Thm: a tuple $\bar{a} \in q(D)$ if and only if there exists $i = 1, \dots, n$ and an **homomorphism** h from ψ_i to D such that $h(\bar{x}) = \bar{a}$

Let's go back to our question:

What **global instance is a good materialization?**

Solutions and Universal Solutions

Consider an information integration system $\langle \mathbf{J}, \mathbf{D} \rangle$, where $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ is a GLAV information integration specification and $\mathbf{D}$ is a **source database**

A **global instance** $\mathbf{K}$ over $\mathbf{G}$ is like a **global database** over $\mathbf{G}$ except that it may contain also **labeled nulls**

- **Definition:** A **global instance** $\mathbf{K}$ is a **solution** for $\mathbf{J}$ w.r.t. $\mathbf{D}$ if $q_S(\mathbf{D}) \subseteq q_G(\mathbf{K})$ for each mapping assertion $\mathbf{m} = \langle q_S, q_G \rangle$ in $\mathbf{M}$, i.e., $(\mathbf{K}, \mathbf{D}) \models \mathbf{M}$
- **Definition:** A **global instance** $\mathbf{K}$ is a **universal solution** for $\mathbf{J}$ w.r.t. $\mathbf{D}$ if $\mathbf{K}$ is a solution for $\mathbf{J}$ w.r.t. $\mathbf{D}$ such that, for each $\mathbf{B} \in \text{sem}(\mathbf{J}, \mathbf{D})$, there exists a **homomorphism** from $\mathbf{K}$ to $\mathbf{B}$

Exercise

Source database: $\mathbf{D} = \{R(a,b), R(b,c)\}$

Mapping: $\mathbf{M} = \{\forall x, y. R(x,y) \rightarrow \exists z. S(x,z) \wedge S(z,y)\}$

Which of the following **global instances** is a **solution** for $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ w.r.t. $\mathbf{D}$? Which is a **universal solution** for $\mathbf{J}$ w.r.t. $\mathbf{D}$?

➤ Symbols x , y , and z denote **labeled nulls**

- $\mathbf{K}_1 = \{S(a,x), S(x,b), S(b,d), S(d,c)\}$
- $\mathbf{K}_2 = \{S(a,x), S(x,b)\}$
- $\mathbf{K}_3 = \{S(a,x), S(x,b), S(a,y), S(y,b), S(b,z), S(z,c)\}$
- $\mathbf{K}_4 = \{S(a,x), S(x,b), S(b,y), S(y,c)\}$
- $\mathbf{K}_5 = \{S(a,x), S(x,b), S(b,x), S(x,c)\}$

Exercise

Source database: $D = \{R(a,b), R(b,c)\}$

Mapping: $M = \{\forall x, y. R(x,y) \rightarrow \exists z. S(x,z) \wedge S(z,y)\}$

Which of the following **global instances** is a **solution** for $J = \langle G, M, S \rangle$ w.r.t. D ? Which is a **universal solution** for J w.r.t. D ?

➤ Symbols x , y , and z denote **labeled nulls**

- $K_1 = \{S(a,x), S(x,b), S(b,d), S(d,c)\}$ - **Solution but not universal**
- $K_2 = \{S(a,x), S(x,b)\}$ - **Not a Solution**
- $K_3 = \{S(a,x), S(x,b), S(a,y), S(y,b), S(b,z), S(z,c)\}$ - **Universal Solution**
- $K_4 = \{S(a,x), S(x,b), S(b,y), S(y,c)\}$ - **Universal Solution**
- $K_5 = \{S(a,x), S(x,b), S(b,x), S(x,c)\}$ - **Solution but not universal**

Naïve Evaluation (recap)

Assume an (*incomplete*) database D and a query q . We denote by $q(D)$ the evaluation of q over D .

The **naïve evaluation** of q over D (written $q^\perp(D)$) is defined as follows:

1. We evaluate q over D as if D was a complete database
2. We discard all the tuples that contain **labelled nulls**

That is, $q^\perp(D)$ is a function that removes from $q(D)$ all the tuples containing at least a **labeled null**

Certain Answers via Universal Solutions

Consider an information integration system $\langle \mathbf{J}, \mathbf{D} \rangle$, where $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ is a GLAV information integration specification and $\mathbf{D}$ is a **source database**

Theorem U: Let $\mathbf{K}$ be a universal solution for $\mathbf{J}$ w.r.t. $\mathbf{D}$. For any CQ q , we have $\text{cert}(q, \mathbf{J}, \mathbf{D}) = q^\perp(\mathbf{K})$

Proof (sketch): We first prove that $\text{cert}(q, \mathbf{J}, \mathbf{D}) \subseteq q^\perp(\mathbf{K})$ by contraposition, i.e. by proving that $\bar{c} \notin q^\perp(\mathbf{K})$ implies $\bar{c} \notin \text{cert}(q, \mathbf{J}, \mathbf{D})$ (by the notion of universal solution). Then, we prove that $q^\perp(\mathbf{K}) \subseteq \text{cert}(q, \mathbf{J}, \mathbf{D})$ by using the following given property: for any CQ $q = \{\bar{x} \mid \phi(\bar{x})\}$ and instance $\mathbf{K}$, $\bar{c} \in q(\mathbf{K})$ iff there is a homomorphism $\mathbf{p}$ from the set of atoms in $\phi(\bar{x})$ to $\mathbf{K}$ such that $\mathbf{p}(\bar{x}) = \bar{c}$. So, suppose $\bar{c} \in q^\perp(\mathbf{K})$. We know that there is a homomorphism $\mathbf{h}$ from the set of atoms in $\phi(\bar{x})$ to $\mathbf{K}$ such that $\mathbf{h}(\bar{x}) = \bar{c}$. Consider any $\mathbf{B} \in \text{sem}(\mathbf{J}, \mathbf{D})$. Since there is a homomorphism $\mathbf{p}$ from $\mathbf{K}$ to $\mathbf{B}$, the composition function $\mathbf{p} \circ \mathbf{h}$ is a homomorphism from the set of atoms in $\phi(\bar{x})$ to $\mathbf{B}$ such that $\mathbf{h}(\bar{x}) = \bar{c}$, and therefore $\bar{c} \in q(\mathbf{B})$. This implies that $\bar{c} \in \text{cert}(q, \mathbf{J}, \mathbf{D})$.

Universal Solutions – Some Observations

The definition of Universal Solutions does not guarantee existence. Indeed, the definition is not **constructive** and does not provide a procedure to compute it.

Universal solutions may not be unique. They are homomorphically equivalent, i.e., for every pair of universal solutions **K** and **K'** there exist homomorphisms h from **K** to **K'** and h' from **K'** to **K**.

For **GAV** mappings, the retrieved **global database** is a **universal solution**!

- It satisfies all the mappings
- It is contained in every model, therefore the identity is the desired homomorphism!

For **GLAV** mappings, we can use the *chase* algorithm(s): a family of algorithms that compute **universal solutions**.

The Chase

The **chase** is a family of algorithms that can compute universal solutions.

- In the literature there are different versions of this algorithm, we will see the simplest.

The chase can be used to perform reasoning tasks over existential rules

Definition: An **existential rule** is a FOL formula of the following form where ϕ and ψ are conjunctions of atoms.

$$\forall \bar{x}. \forall \bar{y}. \phi(\bar{x}, \bar{y}) \rightarrow \exists \bar{z}. \psi(\bar{x}, \bar{z})$$

The chase is based on a set of **intuitions** that we discuss in the next slides.

The Chase – Triggerable Mappings

Assume a GLAV mapping $\mathbf{M}$ and a database $\mathbf{D}$.

Def: A mapping assertion $\langle q_S, q_G \rangle \in \mathbf{M}$ is **triggerable over $\mathbf{D}$** for a tuple of constants $\bar{a}$ (called the trigger) if $\bar{a} \in q_S(\mathbf{D})$

Example: consider a database $\mathbf{D} = \{R(a, a), R(b, c)\}$ and the mapping $\mathbf{M} = \{m_1 : \forall x. R(x, x) \rightarrow \exists y. S(x, y)\}$

We have that m_1 is **triggerable** for $\langle a, a \rangle$ but not for $\langle b, c \rangle$

Triggerable Mappings – Intuition

Intuition 1: to build a solution we need to “materialize” all **triggerable mappings** (for each possible tuple that is a **trigger**)

Ex: $D = \{R(a, a), R(b, c)\}$

$M = \{m_1 : \forall x. R(x, x) \rightarrow \exists y. S(x, y)\}$

We have that m_1 is **triggerable** for $\langle a, a \rangle$

Ex: $K = \{S(a, b), S(a, a), S(c, d)\}$ is a solution.

The Chase – Materializing a GLAV Mapping

Intuition 2: In order to “materialize” a triggerable mapping assertion we can use **variables** (a.k.a. labeled nulls) to feel up the *existentially quantified attributes*

- We do not need to **invent constants** for the *existential variables*!

Ex: $D = \{R(a, a), R(b, c)\}$

$M = \{m_1 : \forall x. R(x, x) \rightarrow \exists y. S(x, y)\}$

We have that m_1 is **triggerable** for $\langle a, a \rangle$

Ex: $K = \{S(a, x), S(a, y), S(c, d)\}$ is a solution.

The Chase – Smallest Materialization

Intuition 3: To compute the most general (**universal**) solution we need to materialize all and **only** the triggerable mappings in the **most general way!**

– To define a universal solution, we cannot add additional information.

Ex: $D = \{R(a, a), R(b, c)\}$

$M = \{m_1 : \forall x. R(x, x) \rightarrow \exists y. S(x, y)\}$

We have that m_1 is **triggerable** for $\langle a, a \rangle$

Ex: $K = \{S(a, x)\}$ is an universal solution.

The Naïve Chase Algorithm

Input: $\langle \mathbf{J}, \mathbf{D} \rangle$, where $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ is a GLAV information integration specification and $\mathbf{D}$ is a **source database**

Output: A **global instance** $\mathbf{K}$ over $\mathbf{G}$ (i.e., an incomplete database over $\mathbf{G}$)

$\mathbf{K} \leftarrow \emptyset$

For each $m : \forall \bar{x}. \phi(\bar{x}) \rightarrow \exists \bar{y}. \psi(\bar{x}, \bar{y})$ occurring in $\mathbf{M}$ do:

 For each $\bar{c} \in q_S(\mathbf{D})$, where $q_S = \{\bar{x} \mid \phi(\bar{x})\}$ do:

 For any $y_i \in \bar{y}$ pick a **fresh labeled null** $n_i \in \bar{n}$;

 Add to $\mathbf{K}$ all the atoms occurring in $\psi(\bar{c}, \bar{n})$;

 End For;

End For;

return $\mathbf{K}$;

We denote by $\mathbf{ch}(\mathbf{M}, \mathbf{D})$ the output $\mathbf{K}$ of the algorithm

The Naïve Chase – Exercise

Consider the source database $\mathbf{D} = \{R(a, b), S(b, c)\}$ and the mapping $\mathbf{M} = \{m_1, m_2, m_3, m_4\}$, where:

- $m_1 : \forall x, y. R(x, y) \rightarrow \exists z. A(x, z) \wedge A(z, y)$
- $m_2 : \forall x, y. S(x, y) \rightarrow \exists z. B(x, z) \wedge B(z, y)$
- $m_3 : \forall x, y. R(x, y) \rightarrow C(y)$
- $m_4 : \forall x, y, z. R(x, y) \wedge S(y, z) \rightarrow A(x, z) \wedge B(x, z)$

Compute $\text{ch}(\mathbf{M}, \mathbf{D})$

The Naïve Chase – Solution

Consider the source database $\mathbf{D} = \{R(a, b), S(b, c)\}$ and the mapping $\mathbf{M} = \{m_1, m_2, m_3, m_4\}$, where:

- $m_1 : \forall x, y. R(x, y) \rightarrow \exists z. A(x, z) \wedge A(z, y)$
- $m_2 : \forall x, y. S(x, y) \rightarrow \exists z. B(x, z) \wedge B(z, y)$
- $m_3 : \forall x, y. R(x, y) \rightarrow C(y)$
- $m_4 : \forall x, y, z. R(x, y) \wedge S(y, z) \rightarrow A(x, z) \wedge B(x, z)$

Compute $\text{ch}(\mathbf{M}, \mathbf{D})$

- $\mathbf{K} = \{ \underbrace{A(a, z_1), A(z_1, b)}_{m_1}, \underbrace{B(b, z_2), B(z_2, c)}_{m_2}, \underbrace{C(b)}_{m_3}, \underbrace{A(a, c), B(a, c)}_{m_4} \}$

The Naïve Chase Algorithm and Universal Solutions

Theorem C: Given $\langle \mathbf{J}, \mathbf{D} \rangle$, where $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ is a GLAV information integration specification and $\mathbf{D}$ is a **source database**, we have that $\mathbf{ch}(\mathbf{M}, \mathbf{D})$ is a **universal solution** of $\mathbf{J}$ w.r.t. $\mathbf{D}$

Proof (sketch): The (complex) proof is based on the following intuitions:

1. Every solution needs to materialize all the *triggered* mappings
2. The use of fresh new variables guarantees homomorphisms
3. Not materializing any extra information guarantees that we create information that needs to occur in all the semantics of $\mathbf{J}$ w.r.t. $\mathbf{D}$

Certain Answers and the Chase Algorithm

Theorem: Given $\langle \mathbf{J}, \mathbf{D} \rangle$, where $\mathbf{J} = \langle \mathbf{G}, \mathbf{M}, \mathbf{S} \rangle$ is a GLAV information integration specification and $\mathbf{D}$ is a **source database**, and CQ q , we have that $\text{cert}(q, \mathbf{J}, \mathbf{D}) = q^\perp(\text{ch}(\mathbf{M}, \mathbf{D}))$

Proof: the statement directly follows from **Theorem U** and **Theorem C**

Query Answering – Recognition Problem

Let **CQ-GLAV** be the class of **conjunctive GLAV mappings**

DEF: UCQ-QA-IIS($\mathcal{L}$) where $\mathcal{L}$ is a class of mappings is the following problem

- **Input:** An I.I.S. $J = \langle G, M, S \rangle$ where G and S are empty theories and M is a mapping in $\mathcal{L}$, a source database D , a UCQ q , and a tuple of constants $\bar{c}$
- **Question:** Is $\bar{c} \in \text{cert}(q, J, D)$?

Thm: UCQ-QA(CQ-GLAV) is **decidable** (actually polynomial in data complexity)

Proof: For a GLAV mapping M and database D , we can compute a **universal solution** K of D w.r.t. M via the **naïve chase** (in polynomial time w.r.t. the size of D) and we can use the **naïve evaluation** to answer the query q over K (in polynomial time w.r.t. the size of D)

The Chase and SQL

Can we use SQL databases to materialize the result of the naïve chase?

- In general no, if we want to use nulls as a variables due to two crucial differences
- We cannot use normal data values for variables

1. **Definition of the result.** SQL is based on Codd nulls, therefore we cannot have repetition of variables.
2. **Query Answering.** SQL is based on 3-valued logics while we want to compute naïve evaluation of queries over the result of the chase.

Credits

The presentations of this module of the course are based on the original slides of Prof. Giuseppe De Giacomo, Prof. Diego Calvanese, Prof. Marco Console, Prof. Maurizio Lenzerini, and Prof. Domenico Lembo.